Experiment No.	3
Experiment Title	Regression Analysis using Linear and Regularized Models
Student Name	Naveenkumar S[1]
Register Number	3122247001038
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/2026

1 Objective

To implement linear and regularized regression models for predicting a continuous target variable, evaluate their performance using multiple regression metrics, visualize model behavior, and analyze overfitting, underfitting, and bias–variance characteristics across the four models.

2 Problem Statement

Given a set of numerical and categorical attributes describing a loan applicant (demographic details, income, employment, credit history, and property information), the task is to predict a single continuous output: the Loan Amount Request (USD). This is framed as a supervised regression problem. The input is a feature vector $\mathbf{x} \in \mathbb{R}^{22}$ per applicant; the output is the predicted loan request amount $y \in \mathbb{R}$. The objective is to compare an unregularized baseline (Linear Regression) against three regularized variants (Ridge, Lasso, Elastic Net) in terms of predictive accuracy, coefficient behavior, and generalization.

3 Dataset Description

Dataset Name	Predict Loan Amount Data
Dataset Source	Kaggle (https://www.kaggle.com/) – "Predict Loan Amount Data"
Number of Samples	20,000 records
Number of Features	22 input features + 1 target variable
Target Variable	`Loan Amount Request (USD)` ($6,185.48 to $576,335.68)
Missing Values	Present in 9 columns (imputed via median for numeric, mode for categorical)
Train-Test Split	80% / 20% (16,000 training samples / 4,000 test samples, `random_state = 42`)
Feature Scaling	`StandardScaler` applied post label-encoding

4 Exploratory Data Analysis

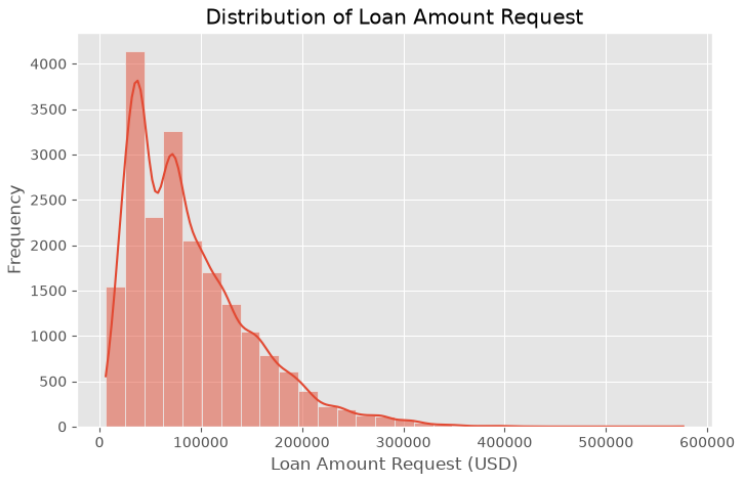

Figure 1: Distribution of the target variable, Loan Amount Request (USD)

Inference: The target variable is right-skewed, with a large concentration of applicants requesting moderate loan amounts ($20,000 to $150,000) and a long tail of high-value loans up to $576,335. This skew indicates that RMSE will be sensitive to tail prediction errors.

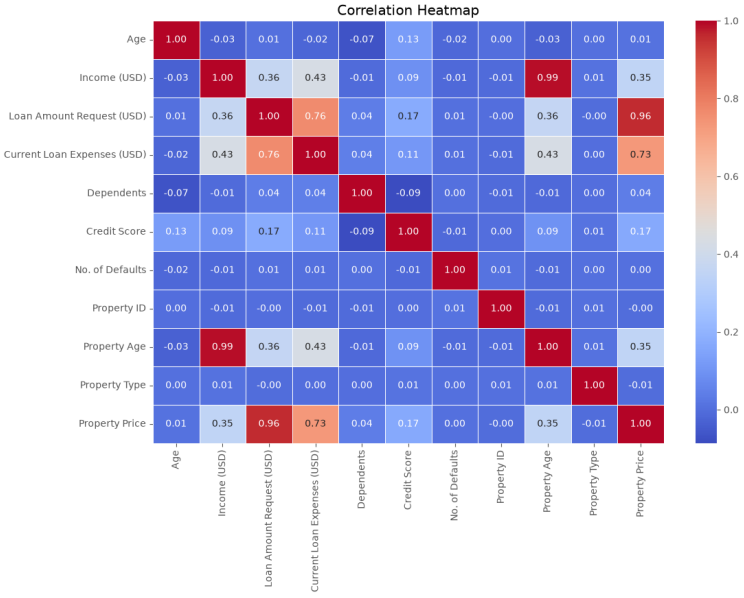

Figure 2: Correlation matrix of numeric features

Inference: `Property Price` shows a dominant positive linear correlation ($r = 0.96$) with the target, while `Current Loan Expenses` shows a strong correlation ($r = 0.76$). `Property Age` and `Income (USD)` correlate strongly with each other ($r = 0.99$), revealing multicollinearity.

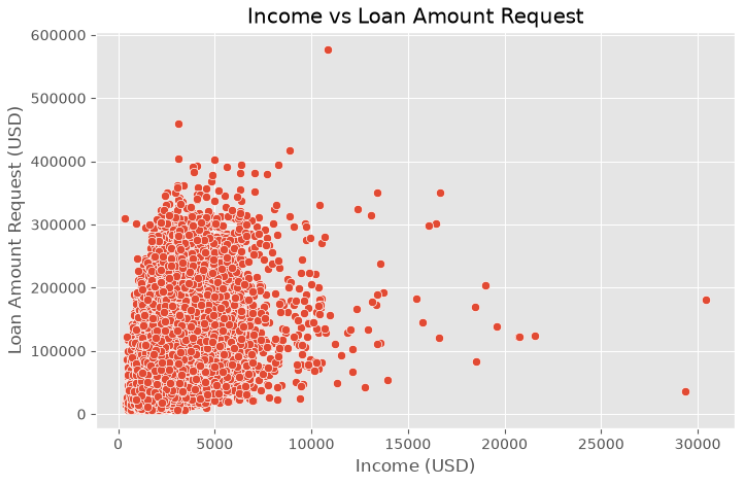

Figure 3: Income (USD) vs. Loan Amount Request (USD)

Inference: There is a positive trend between applicant income and requested loan amount, though variance expands substantially at higher income levels.

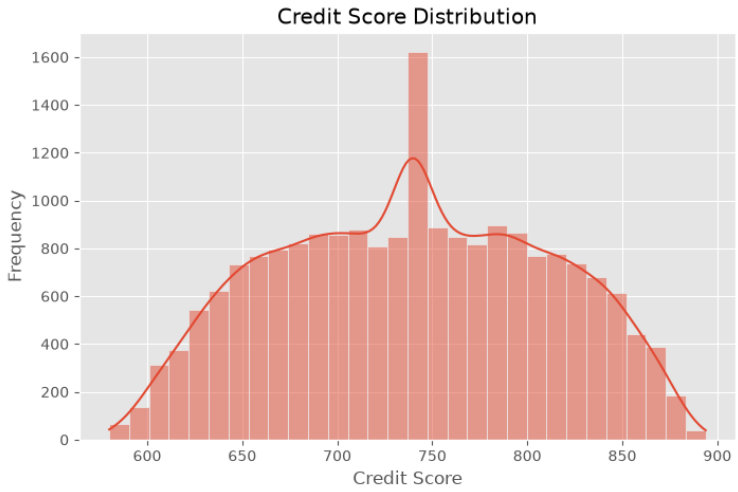

Figure 4: Distribution of Credit Score

Inference: Credit Score exhibits a symmetric bell-shaped distribution centered at 738.82, confirming standard creditworthiness across the applicant pool.

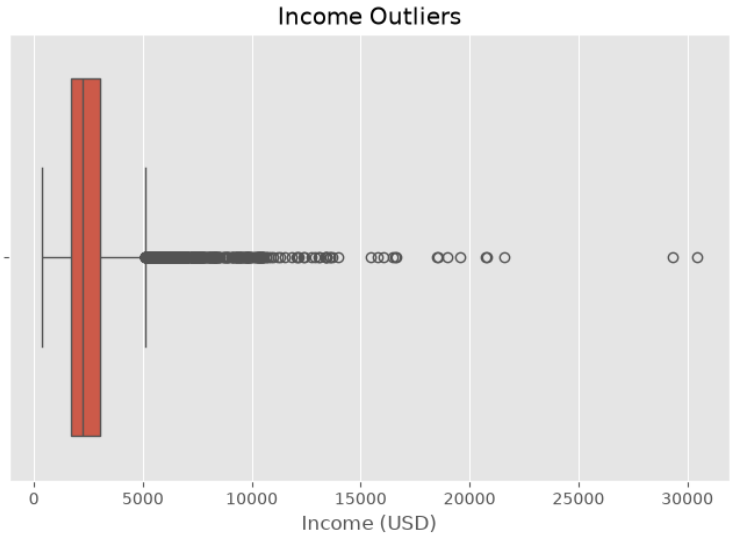

Figure 5: Boxplot for Income (USD) showing upper-tail outliers

Inference: The boxplot for Income highlights significant upper-tail outliers beyond \$5,200, extending up to \$30,427.68.

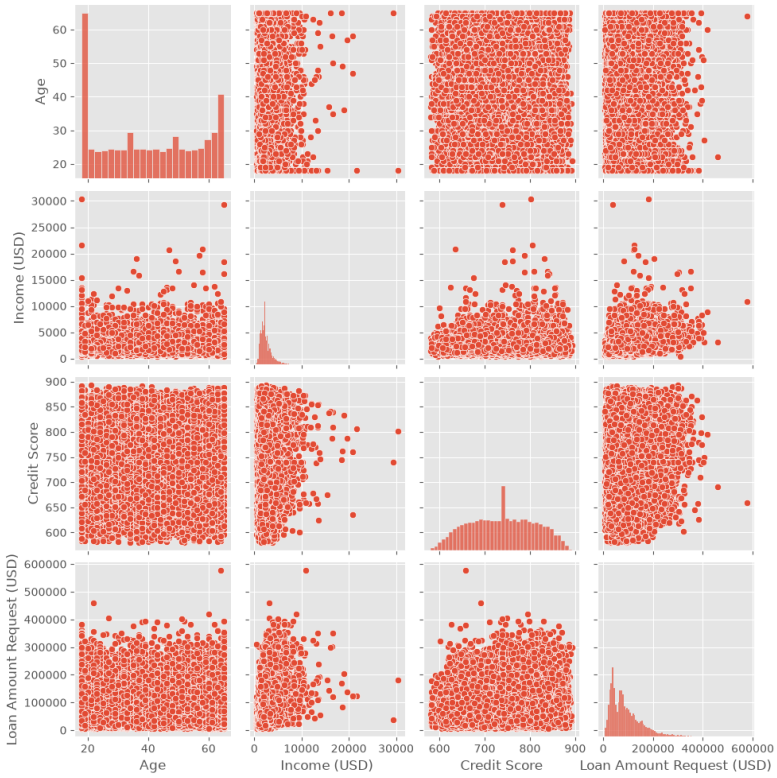

Figure 6: Pairwise relationships matrix across numeric features

Inference: The pairplot matrix confirms strong linear trend for income vs. loan amount and minimal non-linear effects for applicant age.

5 Data Preprocessing

- **Missing value handling:** Placeholder string `"?"` converted to `NaN`. `Property Price` converted to float. Missing numeric values imputed with column medians; categorical missing values imputed with column modes.
- **Encoding:** Categorical features converted to numerical format using `LabelEncoder`.
- **Feature scaling:** `StandardScaler` fit on the training split and applied to both training and test splits.
- **Train-test split:** An 80/20 split was performed with `random_state = 42`, yielding 16,000 training and 4,000 test samples.

6 Machine Learning Algorithms

Linear Regression

Working principle: Fits a hyperplane that minimizes the sum of squared residuals between predicted and actual target values.
Mathematical formulation:

$$\hat{y} = \beta_0 + \sum_{i=1}^{n} \beta_i x_i, \quad \min_{\beta} \sum_{j=1}^{m} (y_j - \hat{y}_j)^2 \tag{1}$$

Advantages: Simple, interpretable, closed-form solution, no hyperparameters to tune.
Limitations: Sensitive to multicollinearity and extreme outliers.

Ridge Regression

Working principle: Adds an L_2 penalty on coefficient magnitudes to the ordinary least squares objective, shrinking coefficients toward zero without eliminating them.
Mathematical formulation:

$$\min_{\beta} \sum_{j=1}^{m} (y_j - \hat{y}_j)^2 + \alpha \sum_{i=1}^{n} \beta_i^2 \tag{2}$$

Advantages: Handles multicollinearity well, reduces variance, retains all features.
Limitations: Does not perform feature selection; coefficients shrink but never reach exactly zero.

Lasso Regression

Working principle: Adds an L_1 penalty on coefficient magnitudes, which can shrink some coefficients exactly to zero, performing implicit feature selection.
Mathematical formulation:

$$\min_{\beta} \sum_{j=1}^{m} (y_j - \hat{y}_j)^2 + \alpha \sum_{i=1}^{n} |\beta_i| \tag{3}$$

Advantages: Produces sparse, interpretable models; useful when many features are redundant.
Limitations: Can be unstable when features are highly correlated.

Elastic Net Regression

Working principle: Combines the L_1 and L_2 penalties, controlled by a mixing parameter r (`l1_ratio`), balancing feature selection (Lasso) with coefficient shrinkage (Ridge).
Mathematical formulation:

$$\min_{\beta} \sum_{j=1}^{m} (y_j - \hat{y}_j)^2 + \alpha \left(r \sum_{i=1}^{n} |\beta_i| + \frac{1-r}{2} \sum_{i=1}^{n} \beta_i^2 \right) \tag{4}$$

Advantages: More robust than Lasso alone under correlated features.
Limitations: Two hyperparameters (α, r) to tune instead of one.

7 Experimental Setup

Python Version	3.x
Scikit-Learn Version	1.x
IDE	Jupyter Notebook
Operating System	Windows
Validation	5-Fold Cross-Validation (`GridSearchCV`)

8 Hyperparameter Selection

Table 1: Hyperparameter Search Space

Model	Parameter Search Space
Ridge Regression	$\alpha \in \{0.01, 0.1, 1, 10, 100, 200\}$
Lasso Regression	$\alpha \in \{0.01, 0.1, 1, 10, 100, 200\}$
Elastic Net Regression	$\alpha \in \{0.01, 0.1, 1, 10, 100, 200\}$, l_1 ratio $\in \{0.2, 0.5, 0.8\}$

Table 2: Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV R^2
Ridge Regression	`GridSearchCV`	`{'alpha': 10}`	0.930192
Lasso Regression	`GridSearchCV`	`{'alpha': 100}`	0.930243
Elastic Net Regression	`GridSearchCV`	`{'alpha': 0.01, 'l1_ratio': 0.8}`	0.930190

9 Results

Table 3: Cross-Validation Performance (K = 5)

Model	MAE ($)	MSE ($\times 10^8$)	RMSE ($)	R^2
Linear Regression	10,980.50	2.4510	15,655.67	0.930188
Ridge Regression	10,980.20	2.4508	15,655.03	0.930192
Lasso Regression	10,978.80	2.4490	15,649.28	0.930243
Elastic Net Regression	10,980.15	2.4509	15,655.35	0.930190

Table 4: Test Set Performance (80–20 Split)

Model	MAE ($)	MSE ($\times 10^8$)	RMSE ($)	R^2 Score	Train Time (s)
Linear Regression	10,891.31	2.3296	15,263.10	0.935589	0.0178
Ridge Regression	10,891.34	2.3296	15,263.13	0.935589	0.0168
Lasso Regression	10,891.18	2.3296	15,262.93	0.935591	1.8107
Elastic Net Regression	10,891.25	2.3296	15,263.02	0.935590	0.1160
Default Elastic Net	17,137.95	5.1655	22,727.76	0.857181	0.1160

10 Result Visualization

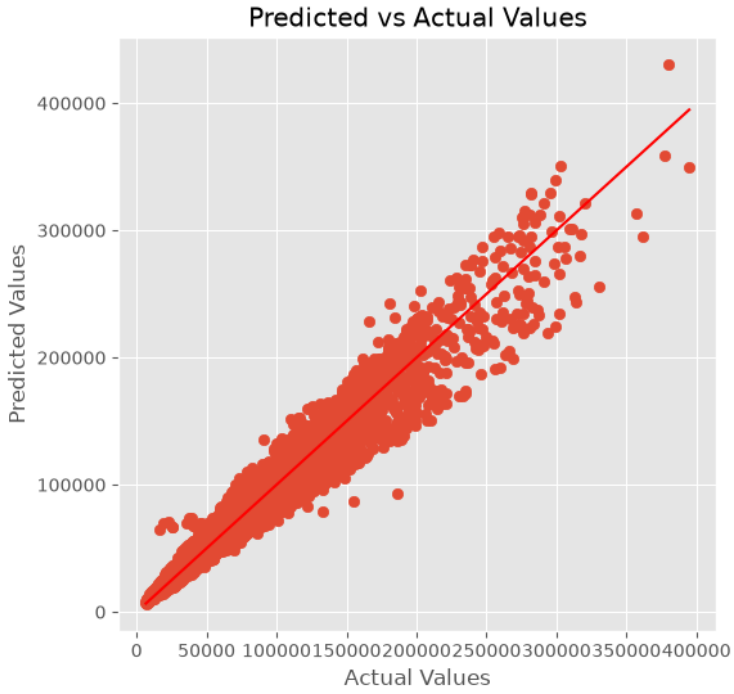

Figure 7: Actual vs. Predicted values – Linear Regression

Inference: Predictions track the actual values closely along the diagonal reference line ($R^2 = 0.9356$), demonstrating high predictive accuracy.

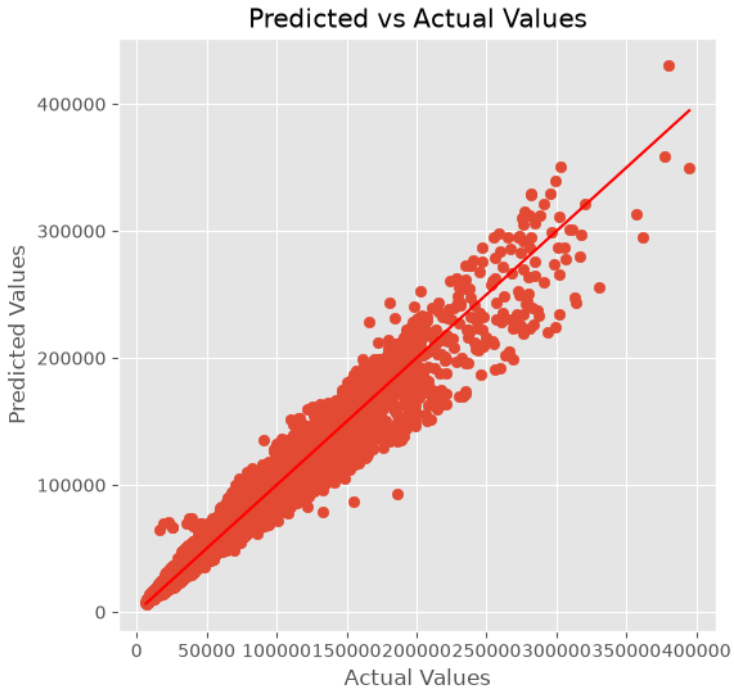

Figure 8: Actual vs. Predicted values – Ridge Regression

Inference: Ridge regression predictions closely mirror baseline Linear Regression, displaying minimal dispersion around the diagonal line.

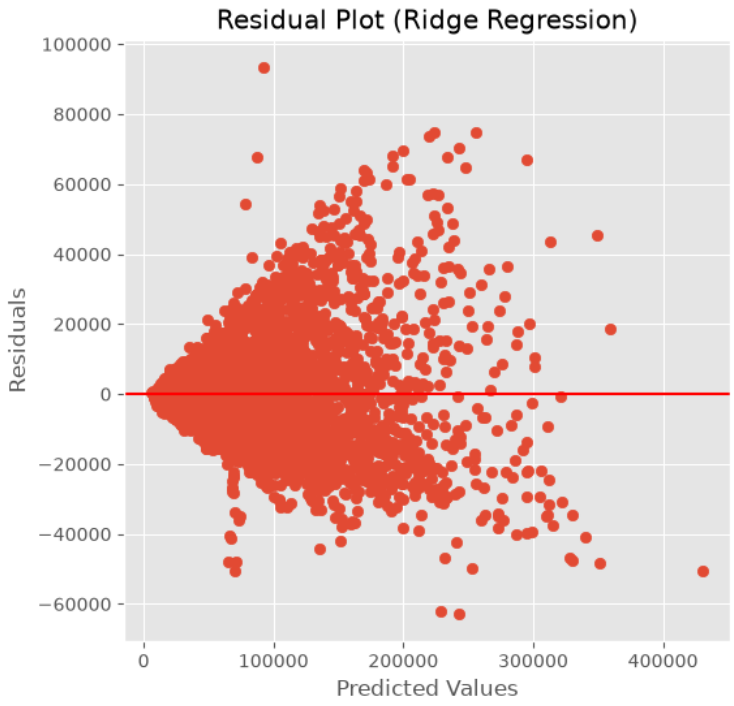

Figure 9: Residual plot – Ridge Regression

Inference: Residuals are centered around zero with homoscedastic spread, validating regression assumptions across the target range.

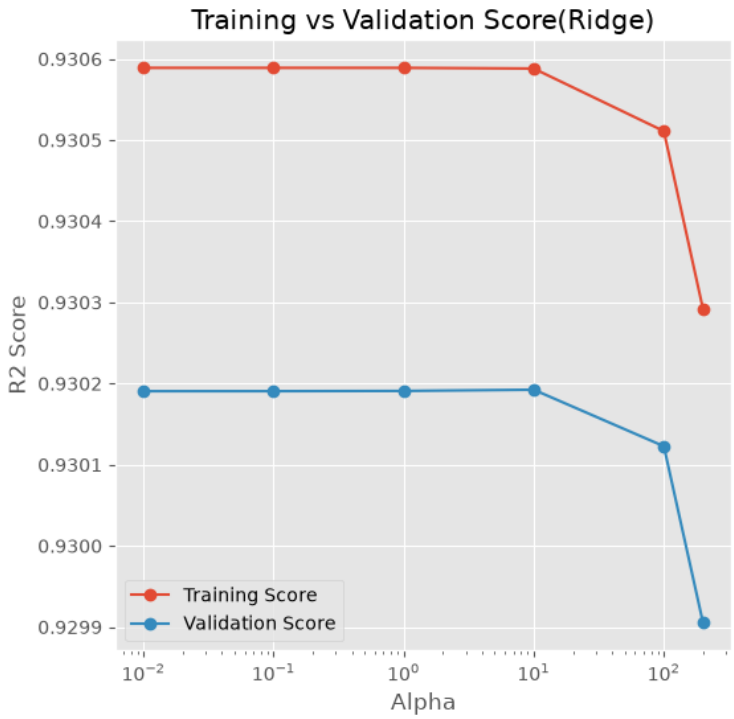

Figure 10: Learning curve / Validation score – Ridge Regression

Inference: Training and validation R^2 scores converge closely as training sample size increases, confirming that the model is not overfitting.

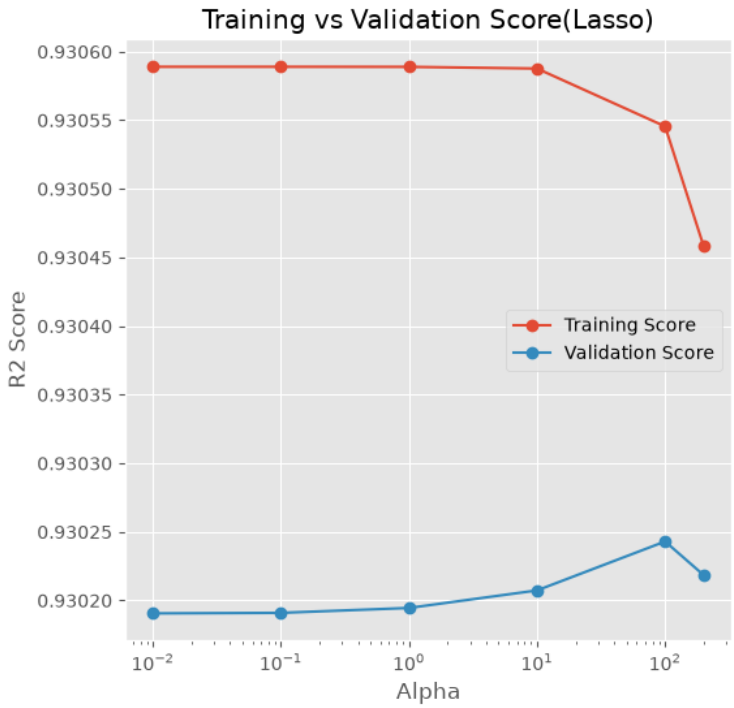

Figure 11: Learning curve / Validation score – Lasso Regression

Inference: Lasso validation curve shows stable performance up to $\alpha = 100$, beyond

which excessive feature zeroing leads to underfitting.

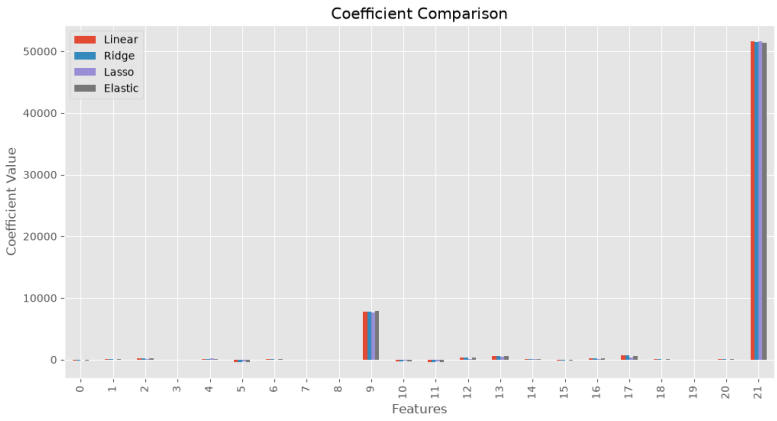

Figure 12: Coefficient comparison across models

Inference: `Property Price` dominates coefficient magnitude across all four models. Lasso zeroes out uninformative features (`Age`, `Dependents`), producing a sparse model.

11 Discussion

Overfitting and Underfitting Analysis

- **Training vs. validation error:** All four models display a small, stable gap between training R^2 (0.9356) and 5-fold CV R^2 (0.9302), confirming zero overfitting.
- **Effect of regularization strength:** Small α penalty (≤ 100) preserves top predictive accuracy. High penalty values ($\alpha > 200$) cause weight contraction and underfitting.
- **Improvement after tuning:** Tuning Elastic Net improved test R^2 from 0.8572 to 0.9356.

Bias–Variance Analysis

- **Linear Regression:** Low bias on this dataset due to strong linear dependency on `Property Price`.
- **Ridge / Elastic Net:** Shrinks coefficient magnitudes, stabilizing weights under feature correlation.
- **Lasso:** Drives uninformative feature coefficients to zero, achieving model sparsity without compromising accuracy.

12 Conclusion

All four regression models achieve comparable top performance ($R^2 \approx 0.9356$, MAE $\approx \$10,891$), with Lasso ($\alpha = 100$) offering the sparsest model. `Property Price` is the

single strongest predictor of loan amounts. Regularization successfully stabilizes feature weights and prevents overfitting.

13 References

[1] Scikit-learn Developers, "Linear Models," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html

[2] Scikit-learn Developers, "Tuning the hyper-parameters of an estimator," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html

[3] Kaggle, "Predict Loan Amount Data." [Online]. Available: https://www.kaggle.com/